ENGINEERING · CONSENT-FIRST MEASUREMENT

Consent Resolution on Higher-Order Load

A device- and browser-agnostic pattern for resolving consent in any client-side template: deny, replay, inject, bridge, ledger — with per-template adapters and the HAR checks that prove it.

A device- and browser-agnostic pattern for resolving consent in any client-side template.

Version 1.0 · 2026-09-04 · Story Story AI

Derived from the `www.crm-sync.dev` Consent Resolution Report (HAR capture, 2026-09-04) and from the shipping implementation in the CRM Sync stack loader. This document generalises that one observed behaviour into a pattern any template can adopt — Shopify Liquid, Webflow, WordPress, Astro, Next, Nuxt, 11ty, AEM/EDS, Salesforce Experience, or a hand-written `index.html`.

---

1. The problem this solves

A consent banner tells you what a visitor chose **once**. It does not tell you what state the page is in on the **next** load, and that gap is where almost every consent defect lives.

The HAR capture made the failure and the fix visible in the same session:

MOMENT	GCS	CLIENT ID	WHAT IT MEANS
Session 1, `form_start`	`G110`	`362792477…` — **temporary**	Analytics denied. Cookieless ping. No persistent identity.
Session 2, after reload	`G111`	`1373221016…` — **persistent**, first issued July 2026	Analytics granted. The real `_ga` cookie was readable all along.

The visitor had already consented. The `_ga` cookie was on the device. But the first page load ran at the **denied default** for its entire lifetime, because nothing replayed the stored choice — the banner only fires on a *new* save. An entire session of behaviour was measured cookielessly against a temporary identifier, and only recovered afterwards by shipping `ga_temp_client_id` for server-side stitching.

That recovery is Google's safety net, not a design. The pattern below removes the need for it.

> **The invariant.** Consent must resolve at **load**, from durable state, before any tag is injected — not at banner interaction. The banner is an *input* to consent state. It is never the *source* of it.

FIGURE 1 · FINDINGS AT A GLANCE

RESOLVES CORRECTLY — THE OBSERVED CHAIN



THREE THAT GENERALISE — AND ONE TO CONFIRM

OBSERVATION	VERDICT	GENERALISED IN
Ad-side Consent Mode keys granted from the very first hit — defensible in the US, a pre-consent leak in the EEA	REVIEW	§6 state model
Merchant Center badge pulls Google Fonts outside every consent gate	REVIEW	§6 third parties
`/signals/log` reflects `Access-Control-Allow-Origin: null` with credentials — a real CORS hole; needs an allow-list	FIX	§5 sandboxes
Only OPTIONS preflights reached the HAR — re-export unsanitized to confirm the POSTs land	VERIFY	§7 conformance

2. The load contract

Five phases, strictly ordered. Phases 0–2 are **synchronous** and must complete inside the first script execution. Phases 3–4 are asynchronous and may complete at any later point in the page's life.

```
PHASE 0   deny everything                      synchronous, before any tag exists
             |
PHASE 1   replay stored decision               synchronous, from durable client state
             |
PHASE 2   inject measurement tags              tags boot into an already-correct state
             |
PHASE 3   bridge: CMP / platform / region      async; may raise or lower consent
             |
PHASE 4   write to the server ledger           async; identity + audit record
```

Everything that follows is an elaboration of those five lines.

Phase 0 — Deny before you load

The consent default block must be the **first executable statement** of the higher-order loader, ahead of any tag, pixel, font, or third-party script. It establishes the state that every later consumer inherits.

Six keys denied; `security_storage` granted. `security_storage` is the strictly-necessary bucket — authentication, fraud prevention, CSRF — and denying it is not a privacy improvement, it is a misdeclaration. (The source report's recommendation to "set all seven keys to denied" should be read as *all six non-essential keys*.)

`wait_for_update: 500` gives an asynchronous CMP a half-second window to raise consent before tags commit to the denied path. It is a grace period for Phase 3

only. **It is not a substitute for Phase 1**, and a template that relies on it in place of a synchronous replay will reproduce exactly the defect in section 1.

Phase 1 — Replay the stored decision, synchronously

This is the phase that is almost universally missing, and the whole point of the pattern.

Read the visitor's persisted decision from durable client state and issue a `consent: 'update'` — never a second `'default'`. `'default'` is a declaration of the pre-decision baseline; re-issuing it after a decision exists misrepresents an informed choice as an absence of one, and downstream platforms model the two differently.

The read must be wrapped in `try/catch`. `localStorage` does not merely return `null` under Safari Private Browsing, a blocked-site-data setting, or a sandboxed context — **the accessor itself throws**. An unguarded read here kills the loader before Phase 2, which fails *open* on some stacks and *silently dark* on others. Both are worse than the defect being fixed.

Phase 2 — Inject the tags

Only now. The measurement library boots into a state that is already correct for this visitor, so its very first hit carries the right `gcs` / `gcd` and, when analytics is granted, the **persistent** client identifier read from the existing cookie. No temporary ID. No stitching. No cookieless first session.

Phase 3 — Bridge to the CMP, the commerce platform, and the region

Consent has more than one authority, and they must not be allowed to disagree:

- **The CMP** (OneTrust, Cookiebot, Osano, or a first-party banner) owns the visitor's expressed choice.
- **The commerce platform** (Shopify Customer Privacy API, and its equivalents) owns whether checkout and platform-native pixels may fire.

- **Region policy** decides whether a decision is even required — a US visitor with no opt-in mandate and an EEA visitor under GDPR reach Phase 3 with different obligations.

A single `resolve()` entry point receives a normalised state and fans it out to every authority in one call. Anything that fans out from more than one place will drift.

Phase 4 — Write the ledger

The browser is a cache. The server is the record.

Safari's ITP caps script-written `localStorage` at seven days of non-interaction; Firefox ETP and Chrome's storage partitioning apply their own eviction. A consent decision that lives only in the browser is a decision you will lose, and losing it is not a measurement inconvenience — it is the loss of the evidence that the decision was ever obtained.

Write the resolved state, the identifier, and a timestamp to a server-side ledger keyed to the authenticated subject. Two rules govern this write:

1. **Bind to the subject, never to the browser.** An anonymous decision has no subject to bind to. Park it locally as *pending* and flush it on login. This is what makes the record defensible.
2. **Write on transitions only.** A CMP fires its wrapper on every page load, including when nothing changed. Comparing against the last-written signature before writing is the difference between an audit trail and audit noise — and re-affirmation churn will make a genuine change unfindable in it.

---

3. Reference implementation

Vendor-neutral, no dependencies, ES5-safe, ~90 lines. It is the same shape as the shipping `stack-loader.js`, with the product-specific names removed. Paste it into any template's `<head>`, above every other script.

```
<script>
(function () {
  // Idempotent: an SPA route change, a duplicated embed, or a CMS block pasted twice
  // must not re-run Phase 0 and wipe a resolved state back to denied.
  if (window.hos && window.hos._init) return;
  var hos = window.hos = window.hos || {};
  hos._init = true;

  var LEDGER = 'https://your-worker.example';   // baked at build time, NOT read from the page
  var STORE_KEY = 'hos_consent';
  var SIG_KEY   = 'hos_consent_sig';

  function store(k, v) { try { if (v === undefined) return localStorage.getItem(k);
                               localStorage.setItem(k, v); } catch (e) { return null; } }

  // ---------- PHASE 0 — deny before anything loads --------------------------
  window.dataLayer = window.dataLayer || [];
  function gtag() { dataLayer.push(arguments); }
  window.gtag = window.gtag || gtag;
  gtag('consent', 'default', {
    ad_storage:              'denied',
    ad_user_data:            'denied',
    ad_personalization:      'denied',
    analytics_storage:       'denied',
    functionality_storage:   'denied',
    personalization_storage: 'denied',
    security_storage:        'granted',   // strictly necessary — never denied
    wait_for_update: 500
  });
```

```
// ---------- PHASE 1 — replay the stored decision, synchronously -----------
function toConsentMode(d) {
  return {
    analytics_storage:       d.analytics  ? 'granted' : 'denied',
    ad_storage:              d.marketing  ? 'granted' : 'denied',
    ad_user_data:            d.marketing  ? 'granted' : 'denied',
    ad_personalization:      d.marketing  ? 'granted' : 'denied',
    functionality_storage:   d.functional ? 'granted' : 'denied',
    personalization_storage: d.functional ? 'granted' : 'denied',
    security_storage:        'granted'
  };
}
try {
  var stored = JSON.parse(store(STORE_KEY) || 'null');
  if (stored) {
    hos._consent = toConsentMode(stored);
    gtag('consent', 'update', hos._consent);   // 'update', never a second 'default'
  }
} catch (e) {}

// ---------- PHASE 2 — inject the tags -------------------------------------
function js(src, onload) {
  var s = document.createElement('script');
  s.src = src; s.defer = true; if (onload) s.onload = onload;
  document.head.appendChild(s); return s;
}
hos.measure = function (id) {
  if (!id || hos._mid) return;
  hos._mid = id;
```

```
  js('https://www.googletagmanager.com/gtag/js?id=' + encodeURIComponent(id));
  gtag('js', new Date());
  gtag('config', id);
};

// ---------- PHASE 3 — the single resolve entry point ----------------------
// Every authority calls THIS. Nothing calls gtag('consent','update') directly.
hos.resolve = function (decision, source) {
  var state = toConsentMode(decision);
  hos._consent = state;
  store(STORE_KEY, JSON.stringify(decision));

  gtag('consent', 'update', state);                                  // measurement plane
  dataLayer.push({ event: 'hos_consent_update', consent: state });   // event bus
  if (hos.platformPrivacy) hos.platformPrivacy(decision);            // commerce plane
  hos.ledger(decision, source || 'banner');                          // record plane
};

// ---------- PHASE 4 — the ledger, on transitions only ---------------------
hos.ledger = function (decision, source) {
  var sig = (decision.analytics ? '1' : '0') + (decision.marketing ? '1' : '0')
          + (decision.functional ? '1' : '0');
  if (store(SIG_KEY) === sig) return;                    // re-affirmation: not an event

  var token = store('auth_token');
  if (!token) { store('hos_consent_pending', JSON.stringify(decision)); return; }  // park

  fetch(LEDGER + '/consent', {
    method: 'POST', keepalive: true,
    headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token },
```

```
      body: JSON.stringify({
        consent_flags: decision,
        method: source,
        consent_id: (window.crypto && crypto.randomUUID)
                      ? crypto.randomUUID() : String(Date.now()),
        timestamp: new Date().toISOString()
      })
    }).then(function () { store(SIG_KEY, sig); }).catch(function () {});
  };

  // Flush a parked anonymous decision the moment a subject exists.
  window.addEventListener('auth-change', function () {
    try {
      var p = JSON.parse(store('hos_consent_pending') || 'null');
      if (p) { hos.ledger(p, 'deferred_login'); localStorage.removeItem('hos_consent_pending');
    } catch (e) {}
  });
})();
</script>
```

Wiring a CMP into it

Any CMP reduces to a category map and one `resolve()` call. OneTrust, as an example:

```
<script>
(function () {
  var prev = window.OptanonWrapper;                 // chain, never replace
  window.OptanonWrapper = function () {
    try { if (typeof prev === 'function') prev(); } catch (e) {}
    var g = String(window.OnetrustActiveGroups || '');
    window.hos.resolve({
      analytics:  g.indexOf('C0002') >= 0,
      functional: g.indexOf('C0003') >= 0,
      marketing:  g.indexOf('C0004') >= 0
    }, 'onetrust');
  };
})();
</script>
```

Chaining rather than replacing the wrapper matters: another integration on the page may already own it, and clobbering it disables their consent handling silently.

---

4. Template adapters

The loader is unchanged in every case. Only the placement differs.

TEMPLATE	PLACEMENT	NOTES
Plain HTML / 11ty / Astro / Hugo	First `<script>` in `<head>` of the base layout	The reference case. Nothing else required.
Shopify (Liquid theme)	Top of `layout/theme.liquid`, above `{{ content_for_header }}`	`content_for_header` injects Shopify's own analytics; the default must precede it. Bridge Phase 3 to `Shopify.customerPrivacy.setTrackingConsent`.
Shopify (Theme App Extension)	An app-embed block with `target: head`	Merchant-toggleable. Note that an app embed loads into `<head>` where DOM-dependent tiers match nothing — keep Phase 0–2 DOM-free, as the reference does.
Webflow	Site-wide custom code, **Head** slot, first block	Webflow's own `webflow.js` loads late and does not contend.
WordPress	`wp_head` at priority `1`	Priority is the whole trick. The default is `10`, which puts you behind most analytics plugins.
Next.js	`next/script` with `strategy="beforeInteractive"` in the root layout	`afterInteractive` is too late — tags will have loaded at denied.
Nuxt	`app.head.script[0]` with `tagPosition: 'head'`	
SvelteKit	`%sveltekit.head%` region of `app.html`, first entry	
AEM / Edge Delivery	`head.html`, before `scripts.js`	
Salesforce Experience Cloud	Head Markup on the site, first block	
SPA route changes	Nothing. Do **not** re-run the loader.	The `_init` guard exists for this. Consent is per-document, not per-route.

Headless and cross-origin storefronts

When the page is not on the commerce platform's own domain, the platform privacy bridge needs the headless form — a public storefront token plus the checkout and storefront root domains — and it only works when the page shares a registrable domain with checkout. On a foreign domain, cross-zone cookies are impossible and the bridge should be **skipped by design** rather than attempted and silently failing. Zone detection belongs on the *hostname*, never on a user-agent string.

---

5. Why this is device- and browser-agnostic

Nothing in the pattern branches on device, browser, or user agent. That is a deliberate property, and these are the rules that preserve it.

No user-agent inspection, ever. A UA string is trivially spoofed and increasingly frozen. Any consent behaviour that varies by UA is a suggestion, not a boundary. Branch on *hostname* (which zone am I in), on *region signal* (does an opt-in mandate apply), and on *stored state* — never on the client's self-description.

Every storage access is guarded. `localStorage`, `sessionStorage`, and IndexedDB accessors throw outright in private modes, sandboxed frames, thumbnail/preview contexts, and under block-site-data settings. Every read and write in the reference is inside `try/catch` and the page renders correctly when all of them return nothing.

Browser storage is a cache; the server is the record. ITP, ETP, and partitioned storage will evict client state on their own schedules. The pattern degrades to "re-prompt the visitor" — never to "silently treat a consented visitor as unconsented and keep measuring", and never to "lose the evidence of consent".

Cookies are never the source of truth. Consent state is read from the loader's own durable key and from the ledger. Third-party cookie deprecation and CHIPS partitioning change nothing about how this pattern resolves.

No `document.write`, no synchronous XHR, no render-blocking third party. Phase 0–2 execute in a few hundred microseconds and inject with `defer`. There is no path on which a slow network turns into an unresolved consent state.

Sandboxed contexts are never trusted. Pixel workers and sandboxed iframes send `Origin: null`. So does every `data:` URL and local file on the internet. Reflecting `null` back with `Access-Control-Allow-Credentials: true` — the defect recorded as finding 4.4 in the source report — allows credentialed cross-origin requests from arbitrary pages. Use an explicit allow-list, and authenticate the sandbox case with a signed request rather than with CORS trust.

Idempotence over instance-counting. The `_init` guard means a duplicated embed, a CMS block pasted twice, or a re-hydrating framework cannot reset a resolved state back to denied.

---

6. The state model

One decision object, three projections. Keep the projections mechanical; put no policy in them.

CATEGORY (CMP)	DECISION KEY	CONSENT MODE V2	COMMERCE PLATFORM	LEDGER
Strictly necessary	— (always on)	`security_storage: granted`	—	implicit
Analytics	`analytics`	`analytics_storage`	`analytics`	`consent_flags.`
Functional / preferences	`functional`	`functionality_storage`, `personalization_storage`	`preferences`	`consent_flags.`
Targeting / advertising	`marketing`	`ad_storage`, `ad_user_data`, `ad_personalization`	`marketing`, `sale_of_data`	`consent_flags.`

Do not grant the ad keys by default. The source capture showed `gcd=13r3q3r3r5l1` on the very first hit — `ad_storage`, `ad_user_data`, and `ad_personalization` all granted while `analytics_storage` was denied. For a US visitor with no opt-in mandate that is defensible. Ship the same default to an EEA or UK visitor and it is a pre-consent advertising-signal transmission. Default all six to denied and let a region-aware Phase 3 raise them; the cost of doing so in a non-opt-in region is one `consent update` call.

Third parties are inside the gate, not beside it

In the source capture a Merchant Center trust badge appeared to load unconditionally from a footer loader, pulling `apis.google.com`, a `google.com` iframe and RPC, `gstatic.com`, and Google Fonts. Reading the loader source settled it the other way: the badge injects `platform.js` only from inside a mount function guarded on marketing consent, and it loaded 250 ms **after** the consent bootstrap because it had been waiting for it. The gate was working.

Keep that correction in view, because the method matters more than the verdict. A HAR cannot distinguish "not gated" from "gated, and consent resolved a moment ago" — both look like a third-party request shortly after page load. Ordering is suggestive, never conclusive. Only two things settle it: the

loader source, or a **reject-all capture**, where a correctly gated resource is simply absent. Treat a third-party request in a consented capture as a question, not a finding.

The rule it illustrates stands regardless: **anything that opens a connection to a third party is inside the consent gate** — badges, fonts, chat widgets, map embeds, video players, review widgets, A/B tools. Gate each on the functional category, or self-host it. The exposure is real even when the widget is trivial: Google Fonts served from `fonts.googleapis.com` was held an unlawful transfer of a visitor's IP address absent consent (LG München I, 3 O 17493/20, 2022). Self-hosting fonts is the cheaper of the two answers and removes the question entirely.

---

7. Conformance

Static assertions

These are cheap to run in CI against the loader source and catch the regressions that matter:

ASSERTION	WHY IT EXISTS
The `consent default` call precedes the tag-injection call **in source order**	Denied-before-load. Ordering is the entire guarantee.
The stored replay uses `'update'`, never a second `'default'`	A re-default misrepresents a decision as an absence of one.
The `_init` guard is the first statement in the IIFE	Duplicate embeds must not reset resolved state.
Every storage access sits inside `try/catch`	Private mode and blocked-storage contexts throw, not return null.
Ledger writes are guarded by a signature comparison	Transitions only; re-affirmation is not an event.
No user-agent test appears anywhere in the consent path	Preserves device-agnosticism by construction.
Anonymous decisions park; server writes require a subject token	Consent provenance is bound to a human, not a browser.

Runtime verification, from a HAR capture

The wire is the only place the pattern is actually true. Capture with **Preserve log on**, from a cold navigation with the Network panel already open — a capture that starts mid-session cannot see the default block, which is the single most important thing to verify.

Decode `gcs` on the first measurement hit. The format is `G1XY`, where `X` is `ad_storage` and `Y` is `analytics_storage`:

GCS	AD_STORAGE	ANALYTICS_STORAGE	VERDICT ON A FIRST HIT
`G100`	denied	denied	Correct for a new visitor pre-decision.
`G110`	granted	denied	**Ad-side leak.** Defaults are not denied.
`G101`	denied	granted	Analytics resolved, ads correctly withheld.
`G111`	granted	granted	Correct **only** if a stored grant was replayed — see below.

Then confirm the resolution actually happened at load:

1. **First hit of a returning consented visitor carries `G111` and the persistent client ID.** If it carries `G110`/`G100` with a fresh client ID and a later hit carries `G111` with `ga_temp_client_id` attached, Phase 1 is missing. That is precisely the defect in section 1.
2. **No `stats.g.doubleclick.net` ping before both ad and analytics storage are granted.**
3. **No third-party host — fonts, badges, widgets — appears before the resolution moment.**
4. **The ledger write is present as a completed 2xx, not only as a `204` preflight.** Seven `OPTIONS` requests returned 204 in the source capture with no matching `POST` recorded. A preflight proves the browser asked permission. It does not prove the write landed.

   When a sanitized export hides the follow-up, the **server side settles it** and is the stronger evidence anyway. In the source case the consent ledger held five hash-chained rows whose `ga_session_id` carried the session component `s1788525894` — byte-identical to the session half of the HAR's own temporary client ID `362792477.1788525894` — written 28 seconds after that session began. The write had landed all along; only the proof was missing from the capture. Join on the identifier the client sent, not on wall-clock time, and a sanitized HAR stops being the last word.

Adversarial captures

Three additional captures, each of which tends to reveal a different class of defect:

- **Reject-all, then reload.** No hit may carry `G111`, and the refusal must be present in the ledger.

- **An opt-in region.** A banner must render, the ad keys must be denied on the first hit, and every gated third party must stay silent.
- **Private browsing / storage blocked.** The page must render, the banner must re-prompt, and no uncaught exception may appear in the console. This is the storage-guard test, and it is the one most often skipped.

---

8. Adoption checklist

☐ Loader placed as the first script in `<head>` for the template (see §4 for the exact slot).

☐ Six non-essential keys default to `denied`; `security_storage` granted.

☐ Stored decision replayed **synchronously** with `'update'` before any tag injects.

☐ Measurement tags injected only after Phase 1.

☐ Exactly one `resolve()` entry point; no direct `gtag('consent','update')` anywhere else.

☐ CMP wrapper chained, not replaced.

☐ Commerce-platform privacy bridge wired, with hostname-based zone detection.

☐ Ledger write bound to an authenticated subject; anonymous decisions parked and flushed on login.

☐ Ledger write guarded by a transition signature.

☐ Every storage access wrapped in `try/catch`.

☐ Third-party fonts self-hosted; every remaining third party gated on the functional category.

☐ CORS allow-list explicit; `Origin: null` never reflected with credentials.

- [ ] Static assertions in CI (§7).
- [ ] Cold-navigation HAR verified in a non-opt-in region, an opt-in region, and under reject-all.

Appendix — parameter reference

Parameters carried on Google measurement hits, as observed in the source capture.

PARAMETER	MEANING
`gcs`	Consent state at hit time: `G1` + ad_storage + analytics_storage (`0` denied, `1` granted).
`gcd`	Full consent-mode descriptor across all types, including whether a default or an update produced the state.
`cid`	Client ID. A value not present in a prior session indicates a temporary, cookieless identifier.
`ep.ga_temp_client_id`	A temporary ID from an earlier cookieless hit, sent for post-hoc stitching. **Its presence is the signature of a missing Phase 1.**
`npa`	Non-personalised ads. `0` = personalisation permitted.
`dma`	Digital Markets Act scope. `1` = EEA consent requirements apply to this hit.
`aip`	Anonymise IP.

Related documents

This document is the **portable pattern** — how any template adopts consent-at-load. Two adjacent documents cover the parts deliberately left out of it, and neither is restated here:

- **JS Execution Order — Challenge & Solution.** The CRM Sync loader's own execution-order contract and the test harness that enforces it. Where §7's static assertions come from, and the place to look for the implementation-specific record rather than the general rule.
- **Consent, Cookies & Preferences — User Guide.** The visitor-facing account: what the banner asks, what each preference controls, how to reset, and where consent history is recorded. Written for the person making the choice, not the engineer wiring the plane.

Source report: `crm-sync-consent-resolution-report.docx` *, Story Story AI, 2026-09-04. Reference implementation: Tier A of the CRM Sync stack loader.*
